BABU BANARASI DAS UNIVERSITY

SCHOOL OF COMPUTER APPLICATIONS

NO SQL PROJECT

Student Management System Using MongoDB

Submitted To	Submitted By
Mr. Harendra Singh Rajpoot	Karan Singh
Department BCA (DS & AI)	**Roll No.** 68
Class BCA DS23	**Academic Session** 2026–2027
Date 25th September 2026	**Project** Student Management System Using MongoDB

MongoDB • NoSQL • CRUD • Query Operators • Sorting • Filtering

1. CREATE DATABASE

The project starts by creating or switching to the Students database. This provides the database context in which the student collection is maintained.

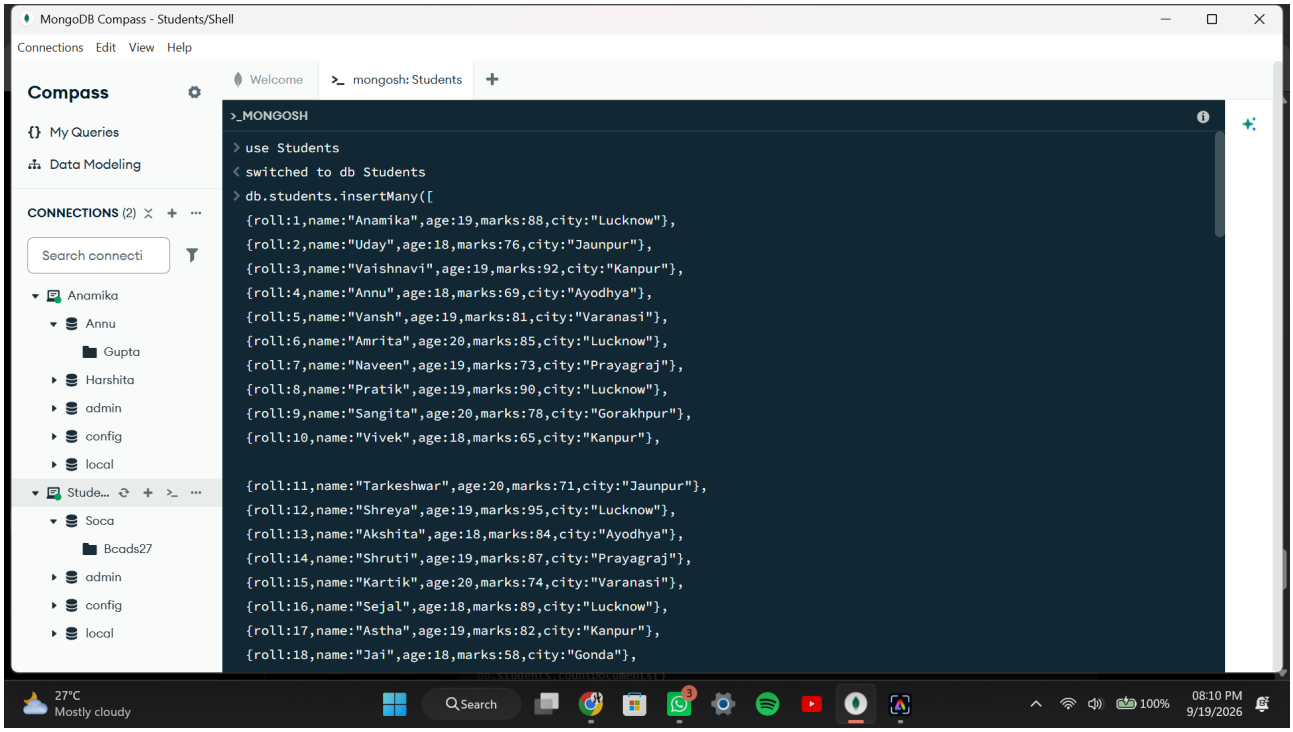


Figure: MongoDB execution screenshot — 1. CREATE DATABASE

2. CREATE COLLECTION

A collection named students is created to store the student documents.

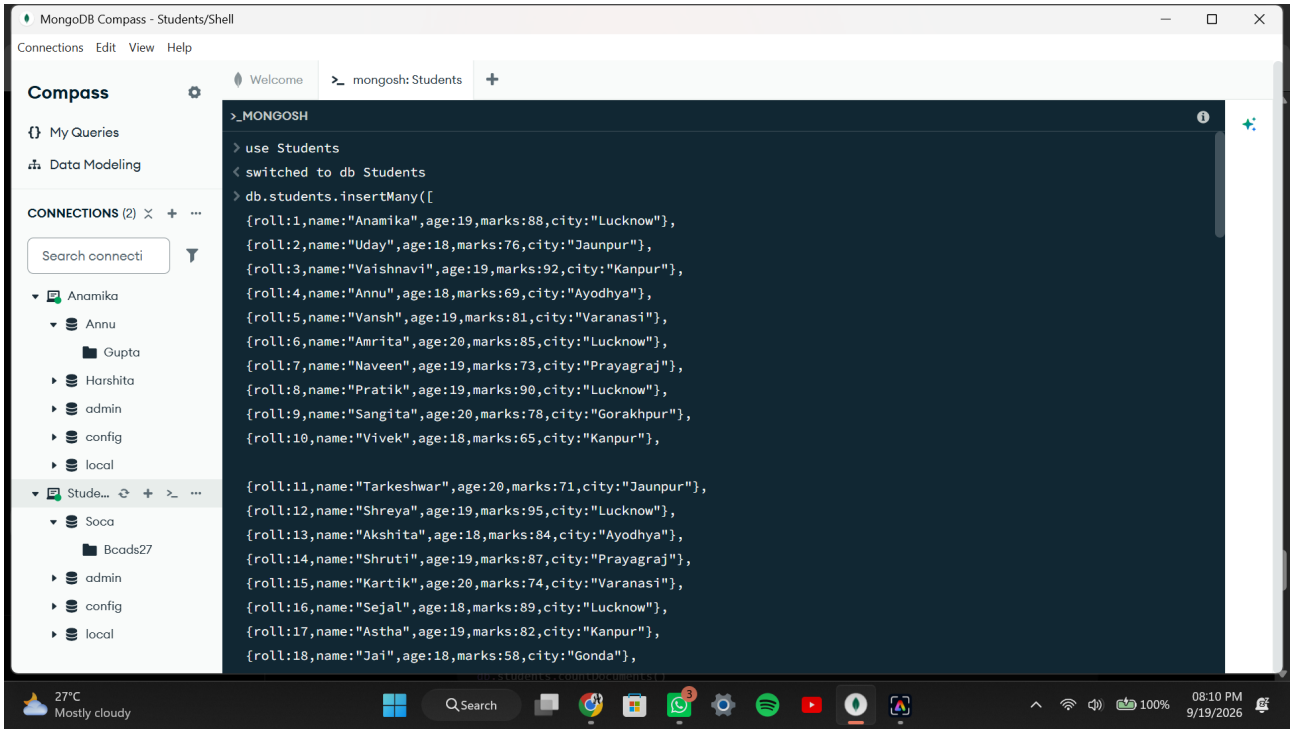


Figure: MongoDB execution screenshot — 2. CREATE COLLECTION

3. INSERT 50 STUDENTS

Multiple student documents are inserted into the students collection. The complete 50-record dataset is reproduced in the dataset section of this report.

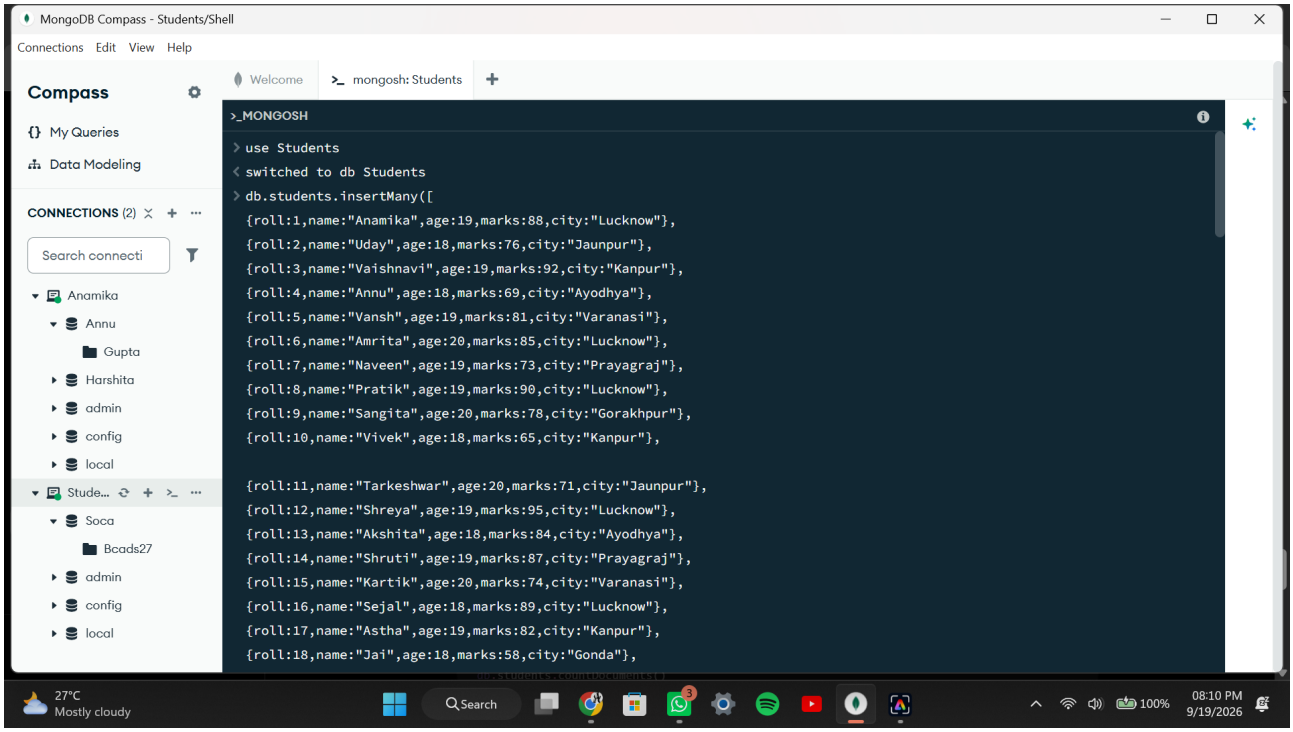


Figure: MongoDB execution screenshot — 3. INSERT 50 STUDENTS

4. DISPLAY ALL STUDENTS

The display operation retrieves documents from the students collection and shows the stored student records.

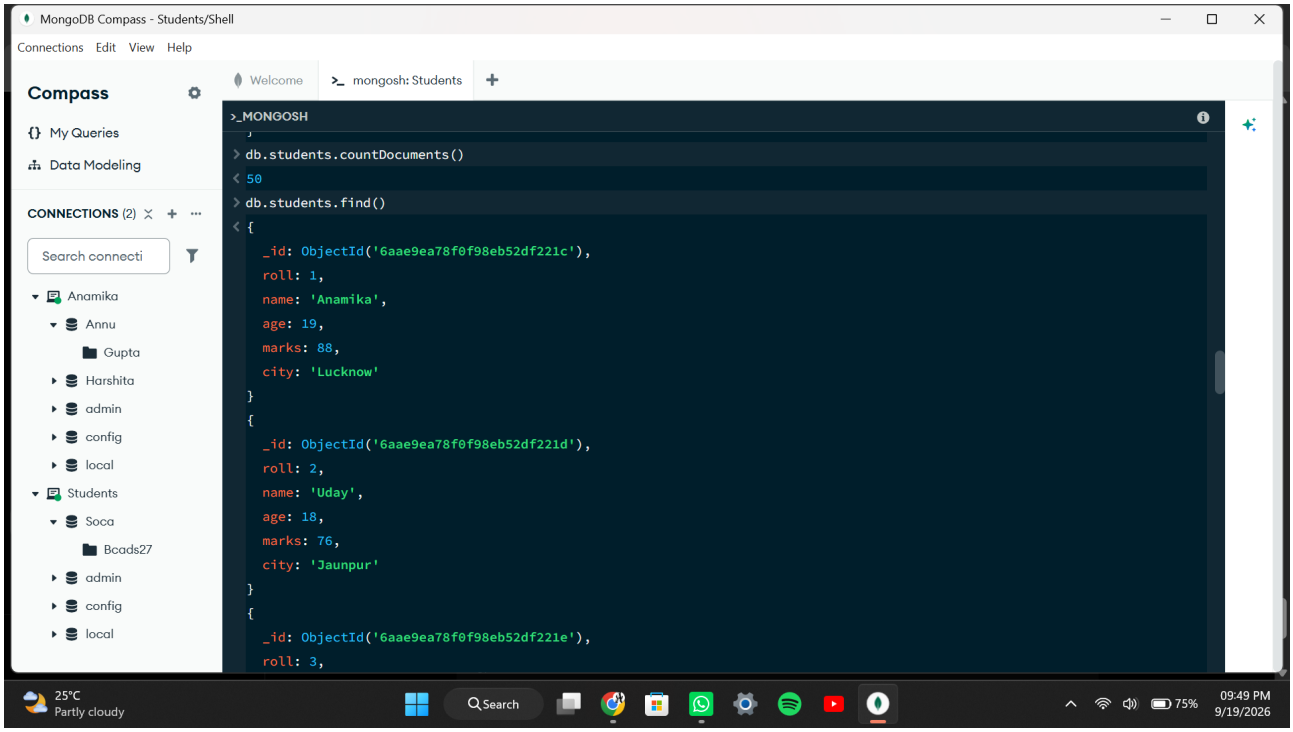


Figure: MongoDB execution screenshot — 4. DISPLAY ALL STUDENTS

5. EQUAL TO — $eq

The $eq comparison operator finds students whose marks are exactly equal to 88.

```
> db.students.find({marks: {$eq: 88}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2236'),
    roll: 27,
    name: 'Ananya',
    age: 19,
    marks: 88,
    city: 'Varanasi'
  }
```

Figure: MongoDB execution screenshot — 5. EQUAL TO $eq

6. GREATER THAN — $gt

The $gt comparison operator finds students whose marks are greater than 85.

```
> db.students.find({marks: {$gt: 85}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df221e'),
    roll: 3,
    name: 'Vaishnavi',
    age: 19,
    marks: 92,
    city: 'Kanpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2223'),
    roll: 8,
    name: 'Pratik',
    age: 19,
    marks: 90,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2227'),
    roll: 12,
    name: 'Shreya',
    age: 19,
    marks: 95,
    city: 'Lucknow'
  }
```

Figure: MongoDB execution screenshot — 6. GREATER THAN $gt

7. LESS THAN — $lt

The $lt comparison operator finds students whose marks are less than 60.

```
> db.students.find({marks: {$lt: 60}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df222d'),
    roll: 18,
    name: 'Jai',
    age: 18,
    marks: 58,
    city: 'Gonda'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df223c'),
    roll: 33,
    name: 'Jai',
    age: 20,
    marks: 55,
    city: 'Ayodhya'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df224a'),
    roll: 47,
    name: 'Aman',
    age: 18,
    marks: 59,
    city: 'Gonda'
  }
> db.students.find({marks: {$eq: 88}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
```

Figure: MongoDB execution screenshot — 7. LESS THAN $lt

8. GREATER THAN OR EQUAL TO — $gte

The $gte comparison operator finds students whose marks are 90 or above.

```
> db.students.find({marks: {$gte: 90}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221e'),
    roll: 3,
    name: 'Vaishnavi',
    age: 19,
    marks: 92,
    city: 'Kanpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2223'),
    roll: 8,
    name: 'Pratik',
    age: 19,
    marks: 90,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2227'),
    roll: 12,
    name: 'Shreya',
    age: 19,
    marks: 95,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df222e'),
    roll: 19,
    name: 'Harshita',
    age: 20,
    marks: 91,
    city: 'Lucknow'
  }
```

Figure: MongoDB execution screenshot — 8. GREATER THAN OR EQUAL TO $gte

9. LESS THAN OR EQUAL TO — $lte

The $lte comparison operator finds students whose marks are 70 or below.

```
> db.students.find({marks: {$lte: 70}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221f'),
    roll: 4,
    name: 'Annu',
    age: 18,
    marks: 69,
    city: 'Ayodhya'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2225'),
    roll: 10,
    name: 'Vivek',
    age: 18,
    marks: 65,
    city: 'Kanpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df222d'),
    roll: 18,
    name: 'Jai',
    age: 18,
    marks: 58,
    city: 'Gonda'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df222f'),
    roll: 20,
    name: 'Omkar',
    age: 19,
    marks: 63,
    city: 'Bareilly'
  }
```

Figure: MongoDB execution screenshot — 9. LESS THAN OR EQUAL TO $lte

10. UPDATE OPERATION

The update operation changes the marks of the student having roll number 5 to 95.

```
>_MONGOSH
}
db.students.updateOne(
  {roll:5},
  {$set:{marks:95}}
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
db.students.find({roll:5})
{
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
}
```

Figure: MongoDB execution screenshot — 10. UPDATE OPERATION

11. DELETE ONE OPERATION

The delete-one operation removes the document matching roll number 10.

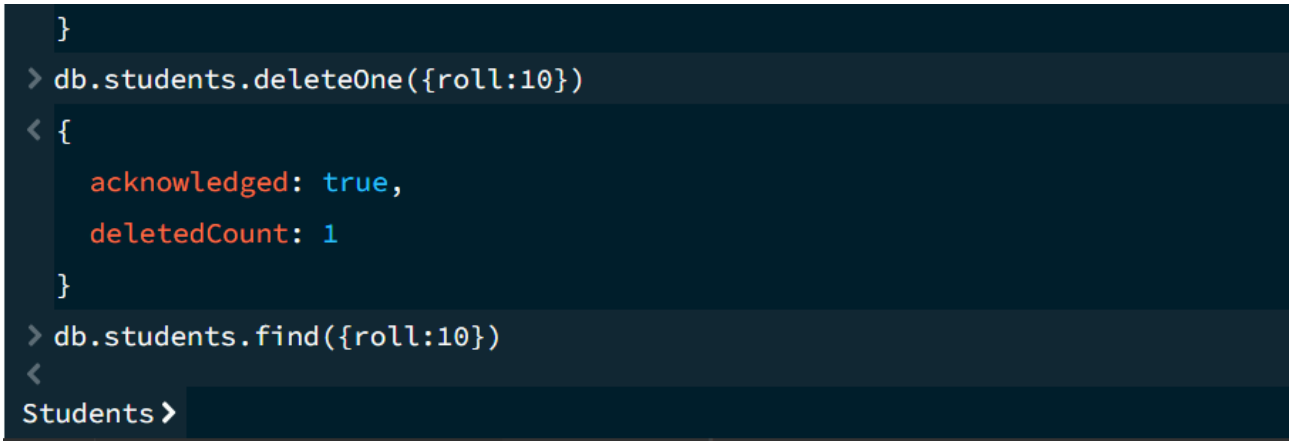


Figure: MongoDB execution screenshot — 11. DELETE ONE OPERATION

12. DELETE MANY OPERATION

The delete-many operation removes all students having marks below 60.

```
> db.students.deleteMany({marks:{$lt:60}})
< {
    acknowledged: true,
    deletedCount: 3
  }
> db.students.find({marks:{$lt:60}})
<
Students >
```

Figure: MongoDB execution screenshot — 12. DELETE MANY OPERATION

13. AND OPERATION — $and

The $and logical operator returns students satisfying both conditions: marks greater than 80 and age equal to 19.

```
>_MONGOSH
> db.students.find({
    $and:[
      {marks:{$gt:80}},{age:19}
    ]
  })
{
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
```

Figure: MongoDB execution screenshot — 13. AND OPERATION $and

14. OR OPERATION — $or

The $or logical operator returns students whose city is either Lucknow or Kanpur.

```
>_MONGOSH
> db.students.find({
    $or:[
      {city:"Lucknow"},{city:"Kanpur"}
    ]
  })
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df221e'),
    roll: 3,
    name: 'Vaishnavi',
    age: 19,
    marks: 92,
    city: 'Kanpur'
  }
  {
```

Figure: MongoDB execution screenshot — 14. OR OPERATION $or

15A. SORT OPERATION — ASCENDING

The sort operation arranges students by marks from lowest to highest. Ascending order uses 1.

```
>_MONGOSH
> db.students.find().sort({marks:1})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df2237'),
    roll: 28,
    name: 'Bhaskar',
    age: 18,
    marks: 61,
    city: 'Gorakhpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2246'),
    roll: 43,
    name: 'Sarika',
    age: 19,
    marks: 62,
    city: 'Ayodhya'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df222f'),
    roll: 20,
    name: 'Omkar',
    age: 19,
    marks: 63,
    city: 'Bareilly'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df224c'),
    roll: 49,
    name: 'Rahul',
    age: 19,
    marks: 64,
    city: 'Bareilly'
  }
```

Figure: MongoDB execution screenshot — 15A. SORT OPERATION ASCENDING

15B. SORT OPERATION — DESCENDING

The sort operation arranges students by marks from highest to lowest. Descending order uses -1.

```
>_MONGOSH
> db.students.find().sort({marks:-1})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df223e'),
    roll: 35,
    name: 'Krishna',
    age: 19,
    marks: 96,
    city: 'Prayagraj'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2227'),
    roll: 12,
    name: 'Shreya',
    age: 19,
    marks: 95,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2220'),
    roll: 5,
    name: 'Vansh',
    age: 19,
    marks: 95,
    city: 'Varanasi'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2235'),
    roll: 26,
    name: 'Anushka',
    age: 20,
    marks: 94,
    city: 'Lucknow'
  }
```

Figure: MongoDB execution screenshot — 15B. SORT OPERATION DESCENDING

16. LIMIT OPERATION

The limit operation displays only the first five documents returned by the query.

```
>_MONGOSH
> db.students.find().limit(5)
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df221d'),
    roll: 2,
    name: 'Uday',
    age: 18,
    marks: 76,
    city: 'Jaunpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df221e'),
    roll: 3,
    name: 'Vaishnavi',
    age: 19,
```

Figure: MongoDB execution screenshot — 16. LIMIT OPERATION

17. SEARCH BY CITY

The city search finds all students whose city is Lucknow.

```
>_MONGOSH
> db.students.find({city:"Lucknow"})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2221'),
    roll: 6,
    name: 'Amrita',
    age: 20,
    marks: 85,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2223'),
    roll: 8,
    name: 'Pratik',
    age: 19,
```

Figure: MongoDB execution screenshot — 17. SEARCH BY CITY

18. COUNT DOCUMENTS

The count operation counts the documents currently present in the collection after the delete operations.

```
>_MONGOSH
  }
> db.students.countDocuments()
< 46
```

Figure: MongoDB execution screenshot — 18. COUNT DOCUMENTS

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how ascending and descending order work, how comparison operators such as $eq, $gt, $lt, $gte and $lte are used, and how logical operators such as $and and $or help in filtering data.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.